

Software Requirements Specification

Multi-Model AI Gateway with Shared Context Memory

Version 0.1 (Draft)

Prepared by: Abhishek

Date: September 15, 2026

Table of Contents

Table of Contents	2
1. Introduction.....	3
1.1 Purpose.....	3
1.2 Scope.....	3
1.3 Intended Audience	3
1.4 Definitions and Abbreviations.....	3
1.5 References.....	3
2. Overall Description.....	3
2.1 Product Perspective.....	4
2.2 Product Functions (Summary)	4
2.3 Development Approach / Phasing.....	4
2.4 Operating Environment	4
2.5 Assumptions and Dependencies	4
3. Functional Requirements	4
3.1 Provider Adapter Layer	4
3.2 Routing and Failover.....	5
3.3 Shared Context Memory	5
3.4 Handoff / Context Restoration.....	5
3.5 Deferred Functionality (Later Phases)	5
4. Non-Functional Requirements.....	6
4.1 Reliability.....	6
4.2 Performance	6
4.3 Extensibility	6
4.4 Observability	6
4.5 Data Retention	6
5. Data Model Summary.....	6
6. Future Enhancements.....	6
7. Open Items / To Be Decided	7

1. Introduction

1.1 Purpose

This document specifies the software requirements for an AI Gateway that provides a unified interface across multiple large language model (LLM) providers (initially OpenAI GPT, Google Gemini, and Anthropic Claude), with automatic failover on rate-limit or token-limit errors, and a shared context memory layer that allows a task to be handed off from one model to another without loss of continuity.

1.2 Scope

The system will act as a middleware layer between client applications and multiple LLM provider APIs. It will:

- Route requests to a configured primary model.
- Detect provider-side failures (rate limits, token limits, timeouts, 5xx errors) and automatically fail over to a secondary configured model.
- Maintain a persistent, provider-agnostic shared memory of task status, recent conversation turns, and key decisions, structured as a linked chain of nodes.
- Reconstruct a context-restoration prompt from this memory so a newly-activated model can continue a task seamlessly.
- Later (post-MVP) support smart/cost-aware routing, a cost/usage dashboard, and provider health checks.

Out of scope for the initial version: fine-tuning, embeddings/RAG pipelines, and support for providers beyond GPT, Gemini, and Claude.

1.3 Intended Audience

Developer(s) building and maintaining the gateway; this document also serves as a personal design reference during incremental development.

1.4 Definitions and Abbreviations

Term	Definition
Gateway	The middleware service that routes requests and manages failover/memory.
Provider	An external LLM API (OpenAI, Gemini, Anthropic).
Failover	Automatic switch to a secondary provider when the primary is unavailable or rate-limited.
Session	A single logical task/conversation tracked across one or more model handoffs.
Node	A single unit of memory in the shared context chain (a turn, decision, or checkpoint).
Checkpoint	A compressed summary node that bounds how far back a traversal must go.
Handoff	The act of transferring an in-progress session from one model to another.

1.5 References

- OpenAI API documentation
- Google Gemini API documentation
- Anthropic Claude API documentation

2. Overall Description

2.1 Product Perspective

The gateway is a standalone backend service. Client applications call the gateway using a single unified interface instead of calling each provider's SDK directly. The gateway is provider-agnostic: adding a new provider requires only a new adapter, not changes to client code or to the memory layer.

2.2 Product Functions (Summary)

- Unified request/response interface across providers (adapter layer).
- Automatic failure detection and failover between providers.
- Persistent, structured shared memory per session (linked-node chain).
- Context-restoration prompt generation for handoffs.
- (Later) cost-aware / capability-aware routing.
- (Later) usage and cost dashboard.
- (Later) periodic provider health checks.

2.3 Development Approach / Phasing

The system will be built incrementally rather than all at once:

- Phase 1 — Connect a single provider (GPT) end-to-end through the gateway.
- Phase 2 — Add a second provider (Gemini) with failover on rate-limit/token-limit errors.
- Phase 3 — Add the shared context memory layer (linked-node chain) and handoff restoration.
- Phase 4 — Add smart/cost-aware routing.
- Phase 5 — Add a cost and usage dashboard.
- Phase 6 — Add scheduled provider health checks.

2.4 Operating Environment

Backend service (language/runtime to be finalized), deployable as a standalone API. Persistent storage for session memory (e.g. Redis or PostgreSQL — to be finalized during Phase 3).

2.5 Assumptions and Dependencies

- Valid API credentials are available for each configured provider.
- Providers' documented error responses (HTTP status codes / error bodies) reliably indicate rate-limit and quota conditions.
- Failover is only guaranteed to be clean when a failure occurs before a response stream has started; mid-stream failures may surface partial output to the client.

3. Functional Requirements

3.1 Provider Adapter Layer

FR-1: The system shall provide a normalized internal request/response format that is independent of any single provider's API schema.

FR-2: The system shall implement a per-provider adapter that translates the normalized format into that provider's request format, and translates provider responses back into the normalized format.

FR-3: Adding a new provider shall require only a new adapter implementation, with no changes to routing, failover, or memory logic.

3.2 Routing and Failover

FR-4: The system shall route each incoming request to a configured primary provider by default.

FR-5: The system shall classify provider errors (rate limit, token/quota limit, timeout, 5xx) using a per-provider error classifier.

FR-6: On a retryable error from the active provider, the system shall automatically retry the request against the next configured provider in priority order.

FR-7: If all configured providers fail for a given request, the system shall return a clear error to the caller rather than retrying indefinitely.

FR-8: The system shall log which provider ultimately served each request, and the reason for any failover.

3.3 Shared Context Memory

FR-9: The system shall maintain one memory chain per session, identified by a unique session ID.

FR-10: Memory shall be structured as a sequence of nodes, where each node stores a reference to the ID of its immediate predecessor (a linked-list-style chain) rather than as a single flat, unordered log.

FR-11: Each node shall record at minimum: a node ID, a predecessor ID, a node type (turn, decision, or checkpoint), content, the model that produced it, the task status at that point, and a timestamp.

FR-12: The system shall support decision nodes that are addressable independently of the surrounding conversation, so that the reasoning behind a specific choice can be retrieved directly rather than re-derived from a summary.

FR-13: The system shall support checkpoint nodes containing a compressed summary of all preceding nodes, so that traversal for a handoff can stop at the nearest checkpoint instead of walking the entire chain.

FR-14: The system shall support branching: if a new model begins work from a point earlier than the current chain tip (e.g. after a failed attempt), its new node(s) shall reference that earlier node as predecessor, producing a fork rather than overwriting or corrupting the existing chain.

FR-15: The system shall track a current "head" node per session, representing the most recent state of that session.

3.4 Handoff / Context Restoration

FR-16: On failover or explicit provider switch, the system shall traverse the memory chain backward from the head node — stopping at the nearest checkpoint or the start of the chain — to assemble the context needed for handoff.

FR-17: The system shall generate a context-restoration prompt summarizing task status, relevant prior decisions, and the next intended step, and shall pass this to the newly-activated provider in place of (or in addition to) raw conversation history.

FR-18: The restoration prompt shall be constructed in the normalized internal format and translated by the relevant provider adapter, so restoration logic remains provider-agnostic.

3.5 Deferred Functionality (Later Phases)

- Smart routing: select a provider based on cost, capability, or task complexity rather than fixed priority order.

- Cost dashboard: track and display token usage and estimated cost per provider/session.
- Health checks: periodically probe each provider and adjust routing priority based on observed availability/latency.

4. Non-Functional Requirements

4.1 Reliability

NFR-1: Failover shall occur automatically without requiring the client application to detect the failure or reissue the request itself, except in the documented mid-stream-failure case.

4.2 Performance

NFR-2: Memory chain traversal for a handoff shall be bounded by checkpoint frequency, so that traversal cost does not grow unbounded with total session length.

4.3 Extensibility

NFR-3: The architecture shall allow new providers, new routing strategies, and new memory node types to be added without breaking existing sessions or requiring a data migration for past sessions where reasonably possible.

4.4 Observability

NFR-4: The system shall log, at minimum, which provider served each request, all failover events with cause, and all handoff events with the session ID involved.

4.5 Data Retention

NFR-5: Session memory shall persist for the lifetime of the task and shall be retrievable for as long as the session is considered active (exact retention policy to be finalized).

5. Data Model Summary

Illustrative shape of a memory node (exact field names/types to be finalized during implementation):

Field	Description
node_id	Unique identifier for this node.
prev_id	ID of the causal predecessor node (forms the chain).
session_id	ID of the session this node belongs to.
type	turn decision checkpoint
content	Text content of the turn, decision, or checkpoint summary.
model_used	Which provider/model produced this node.
status_at_this_point	Task status snapshot: in_progress blocked done
references	(decision nodes) IDs of related nodes this decision draws on.
timestamp	When this node was created.

6. Future Enhancements

- Cost-aware and capability-aware smart routing.
- Real-time cost/usage dashboard.
- Scheduled provider health checks feeding back into routing priority.

- Support for additional providers beyond GPT, Gemini, and Claude.

7. Open Items / To Be Decided

- Backend language/framework and persistent storage technology (e.g. Redis vs. PostgreSQL) for the memory layer.
- Exact checkpoint frequency/trigger (fixed node count vs. every model switch vs. time-based).
- Session memory retention policy.
- Format and criteria for automatic extraction of "key decision" nodes from raw model output.